JAVA PROGRAM REPORT

Valid Parentheses Using Stack

1. Objective / Aim

To check whether the brackets in an input string are balanced and correctly arranged using a stack.

2. System Requirements

Hardware: Standard PC/laptop, keyboard, monitor, and sufficient memory/storage.

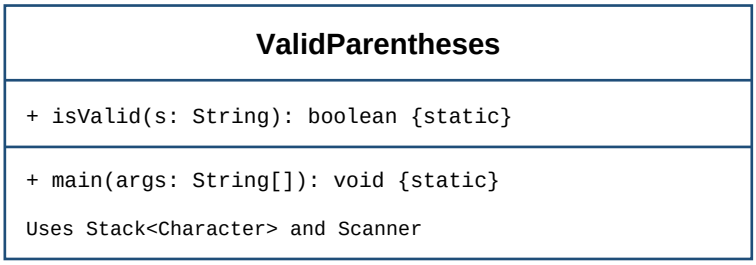
Software: Windows/Linux/macOS, Java Development Kit (JDK), and a Java IDE or command prompt/terminal.

3. Algorithm / Logic Description

1. Read a string from the user.
2. Create an empty stack to store opening brackets.
3. Scan the string one character at a time.
4. Push each opening bracket (, {, or [onto the stack.
5. For a closing bracket, return false if the stack is empty; otherwise pop the top bracket.
6. If the popped opening bracket does not match the closing bracket, return false.
7. After scanning all characters, return true only if the stack is empty.
8. Print whether the string is valid and close the Scanner.

4. UML Class Diagram

The ValidParentheses class contains the isValid method for checking bracket pairs and the main method for user input and output.



5. Source Code

```
import java.util.Scanner;
import java.util.Stack;

public class ValidParentheses {
    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            // Push opening brackets onto the stack
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else {
                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                // Check whether the brackets match
                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {
                    return false;
                }
            }
        }

        return stack.isEmpty();
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the string: ");
        String s = sc.nextLine();

        if (isValid(s)) {
            System.out.println("Output: Valid");
        } else {
            System.out.println("Output: Not Valid");
        }
        sc.close();
    }
}
```

6. Compilation and Execution Commands

```
javac ValidParentheses.java
java ValidParentheses
```

7. Console Output Screenshot

Sample runs from the compiled program:

```
C:\Java-Projects> javac ValidParentheses.java
C:\Java-Projects> java ValidParentheses
Enter the string: ({[]})
Output: Valid
C:\Java-Projects> java ValidParentheses
Enter the string: ([)]
Output: Not Valid
C:\Java-Projects> java ValidParentheses
Enter the string: (((
Output: Not Valid
```

8. Explanation

• **Stack:** Stores opening brackets until matching closing brackets are found.

• **push():** Adds an opening bracket to the top of the stack.

• **pop():** Removes the most recent opening bracket to check the matching pair.

• **isEmpty():** Checks whether the stack has any unmatched opening brackets.

• **isValid():** Returns true for balanced brackets and false for mismatched or unclosed brackets.

Code check: The code compiles successfully. The sample inputs ({[]}) and ([)] produce Valid and Not Valid respectively. The method assumes the input contains only bracket characters; any other character is treated as a closing-bracket case.